

## Lab 4: Hardware Data Prefetcher Design and Analysis

### Introduction

In this lab, I implement a GHB-based stride hardware data prefetcher. In a later step I improve the design by making it aware of system memory bandwidth constraints. Finally I create my own hybrid prefetcher, which chooses between a best-offset-prefetcher (Michaud 2016) and the previous strided GHB prefetcher.

### Warm Up: Running the No-prefetching Baseline

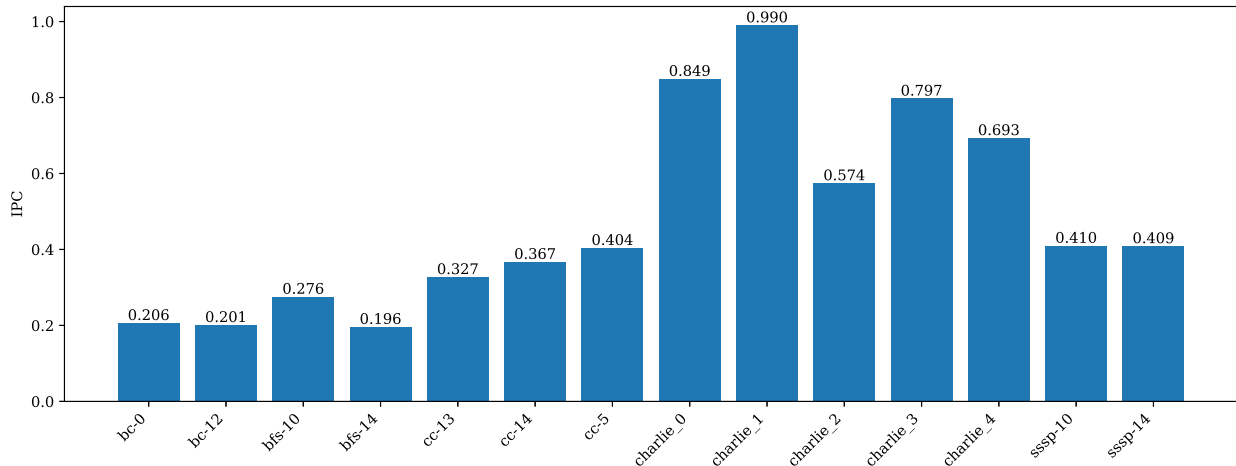


Figure 1: Baseline performance of the system without an L2C prefetcher on real-world CPU traces

The hardware prefetcher designs are evaluated on CPU traces collected from fourteen workloads sourced from graph analytics benchmarks and data-center workloads. Each workload was run for 50 million cycles following 10 million cycles of initial warmup using the Champsim microarchitectural simulator (Gober et al. 2022). The simulated system has a single out-of-order core and a single channel of DRAM running at a data rate of 4800 MT/s (“full BW”).

In fig. 1 is a plot of the instructions-per-cycle (IPC) for each workload. The `charlie_1` trace resulted in the highest IPC value, while the `bfs-14` trace got the lowest IPC value.

### Task 1/4: Implementing GHB-based Stride Prefetcher

#### Implementation

In Task 1 I implemented a GHB-based stride prefetcher (Nesbit and Smith 2004) for the L2C as closely as possible to the original specification. I took special care to accurately replicate the bit-width of tags in the Index Table (IT) and pointers in the Global History Buffer (GHB). However in my own testing, I found that the IPC results for the exact replication are identical to those with a more relaxed replication of the paper with wider bit-widths (tbl. 1). First, this shows that the hardware budget proposed in the paper provides performance equal to an expensive prefetcher, but with a fraction of the resources. Second, this means that my replication of the original prefetcher isn’t sensitive to the specific sizings of the table entries.

Table 1: Per-trace IPC for GHB PC/CS with 20b and 56b wide IT tags (1 cpu, full BW)

| trace   | 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     | 9     | 10    | 11    | 12    | 13    | 14    |
|---------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 20b IPC | 0.229 | 0.221 | 0.362 | 0.293 | 0.415 | 0.476 | 0.532 | 0.864 | 1.014 | 0.582 | 0.809 | 0.704 | 0.548 | 0.554 |
| 56b IPC | 0.229 | 0.221 | 0.362 | 0.293 | 0.415 | 0.476 | 0.532 | 0.864 | 1.014 | 0.582 | 0.809 | 0.704 | 0.548 | 0.554 |

The per-workload IPC speedups, prefetching accuracies and prefetches per thousand instructions (PPKI) caused by the strided GHB prefetcher in the L2C are plotted in fig. 2. The IPC speedup is simply the ratio of the IPC measured with the prefetcher and the IPC without the prefetcher. Next, the accuracy is defined as the ratio of

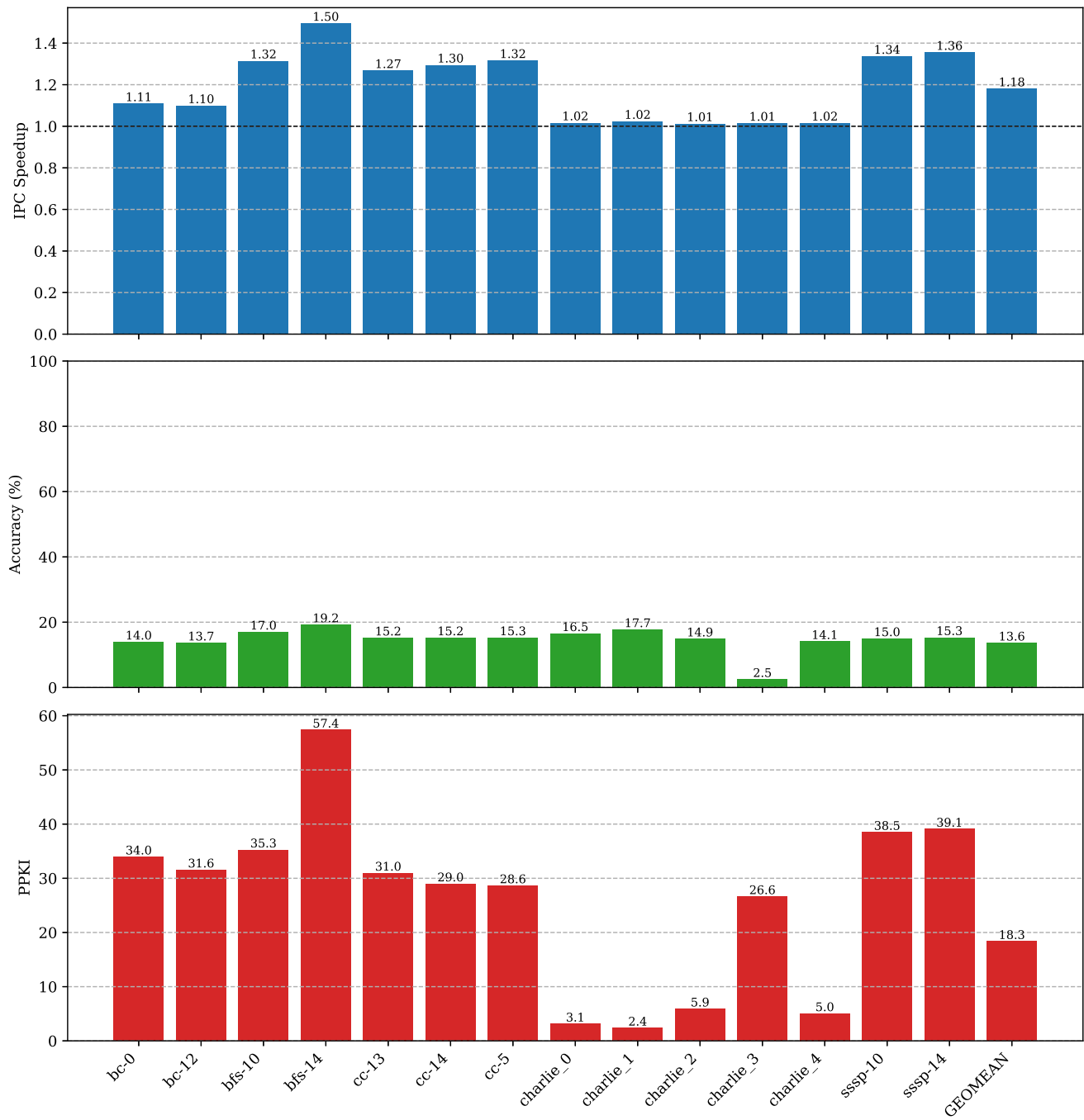


Figure 2: Task 1: IPC speedup, prefetch accuracy, and prefetches per thousand instructions (PPKI) of strided GHB prefetcher (1 CPU, full BW)

prefetches reported by Champsim logs as being “useful” (`l2c_prefetch_useful`) and the total number of issued prefetches (`l2c_prefetch_issued`).

## Results

The utility of the prefetcher is highly heterogenous for the different types of workloads. As explained in the following paragraphs, this indicates that the strided GHB prefetcher is only useful for workloads with access patterns that play to its strengths. Notice that the IPC speedup is always greater than 1, implying that the prefetcher gains a strict performance increase.

**GAP Workloads** `bfs-14` was the workload with the largest IPC speedup. It was also the workload with the largest amount of prefetches issued, at the highest accuracy. The large number of prefetches issued indicate that the GAP workloads (graph analytics) can benefit a lot from a strided GHB prefetcher, considering they are most likely traversing neighboring nodes lists.

**Charlie Workloads** `charlie_2` got the lowest IPC speedup of all workloads. With the exception of `charlie_3`, very few prefetches were issued for this group of workloads, leading to very low IPC speedups overall. For `charlie_3`, the number of issued prefetches was much higher, but with very poor accuracy. This indicates that the charlie workloads (sourced from Google data centers) feature cache accesses that don’t suit a constant-stride prefetcher’s profile. Possibly these workloads resemble pointer-chasing or random accesses.

## Task 2/4: Prefetching with Limited Main Memory Bandwidth

In this task, I investigate the effects of limited main memory bandwidth on the strided GHB prefetcher. The system with limited main memory bandwidth (“limited BW”) is identical to the system with full BW except that the main memory bandwidth is 800 MT/s, a sixth of the full BW.

## Results

The performance of the strided GHB prefetcher in a memory bandwidth limited system is plotted in fig. 3. There are a few observations to make.

- First, the IPC speedups measured in the limited BW system are strictly lower than those from the unconstrained full BW system.
- Next, the IPC speedup on the `charlie_3` workload is below 1, indicating a performance decrease. It achieves the lowest IPC speedup of all workloads. This is most likely due to the combinations of large PPKI, low accuracy and limited bandwidth availability. The largest IPC speedup is observed in the `bfs-14` workload.
- Finally, the largest IPC speedup decreases are on the `sssp-10` and `sssp-14` traces with  $-0.104$  and  $-0.106$  respectively. The smallest IPC speedup decrease is on the `cc-13` workload with  $-0.002$ .

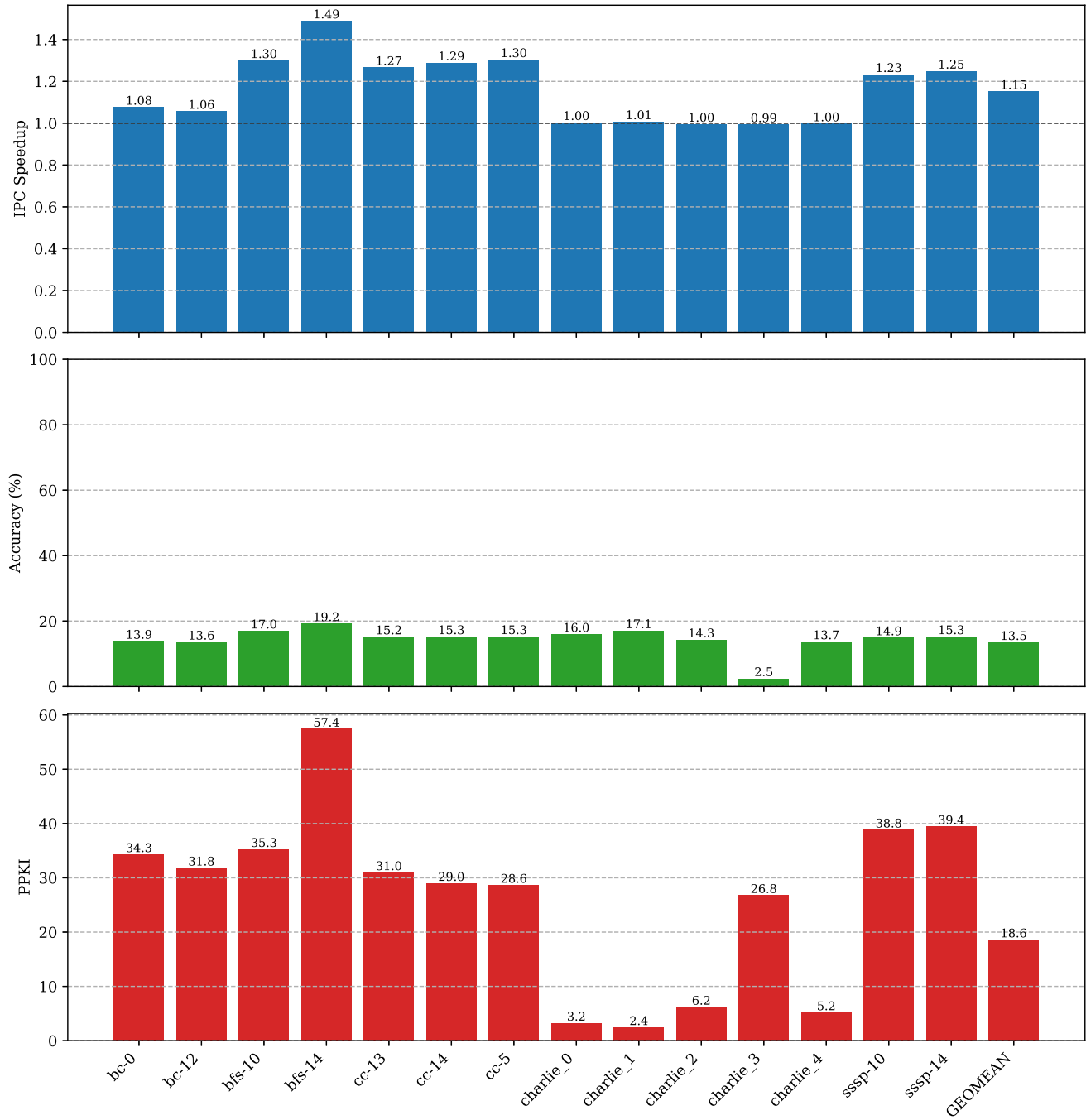


Figure 3: Task 2: IPC speedup, prefetch accuracy, and prefetches per thousand instructions (PPKI) of strided GHB prefetcher (1 CPU, limited BW)

### Task 3/4: System-Aware Prefetcher Design

As suggested in the task description, the strided GHB prefetcher performs worse in a bandwidth-limited system because speculative prefetches consume valuable memory bandwidth. Task 3 therefore introduces system feedback: the prefetcher adaptively throttles its aggressiveness using a heuristic scheme described in the assignment.

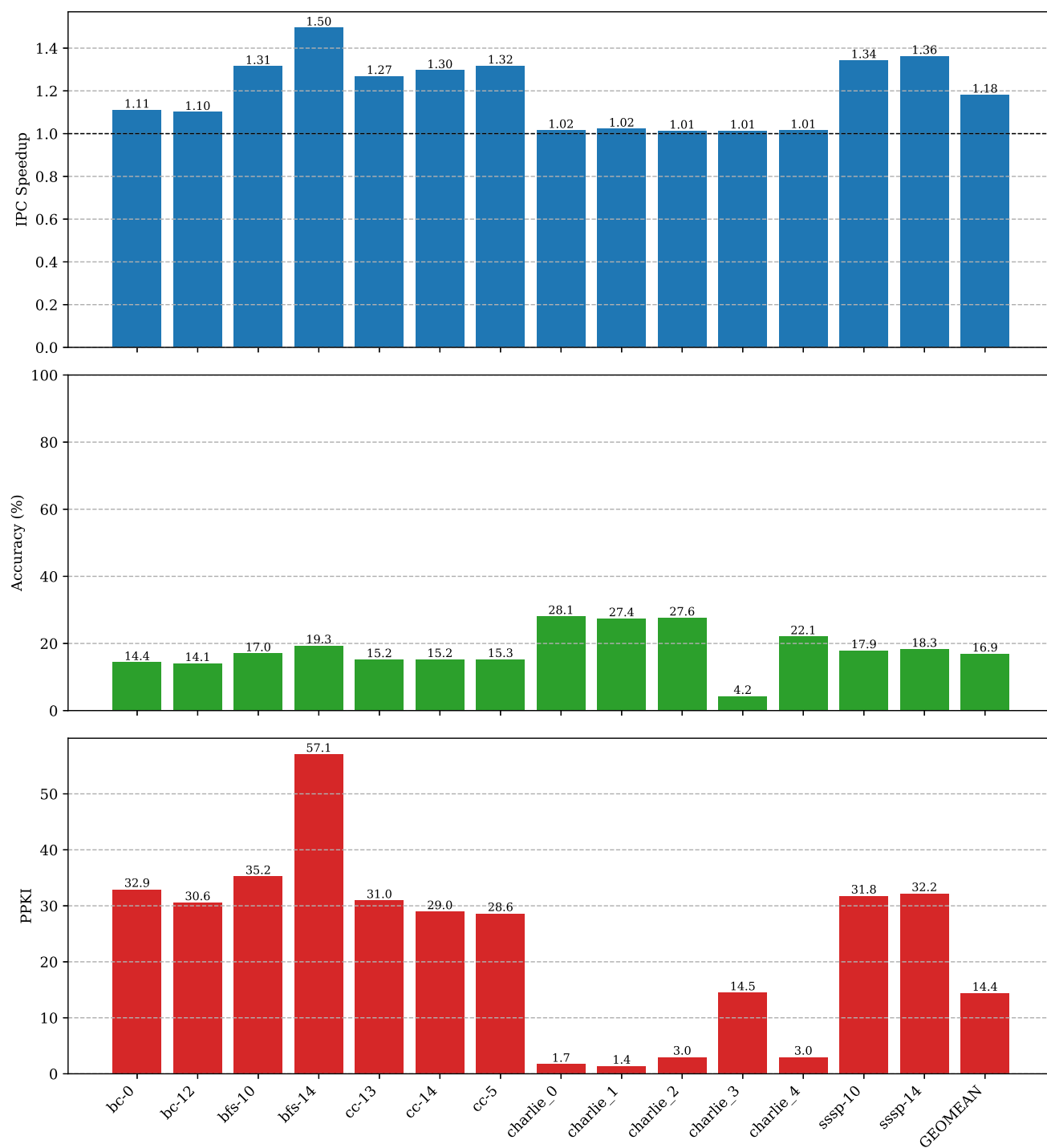
#### Results

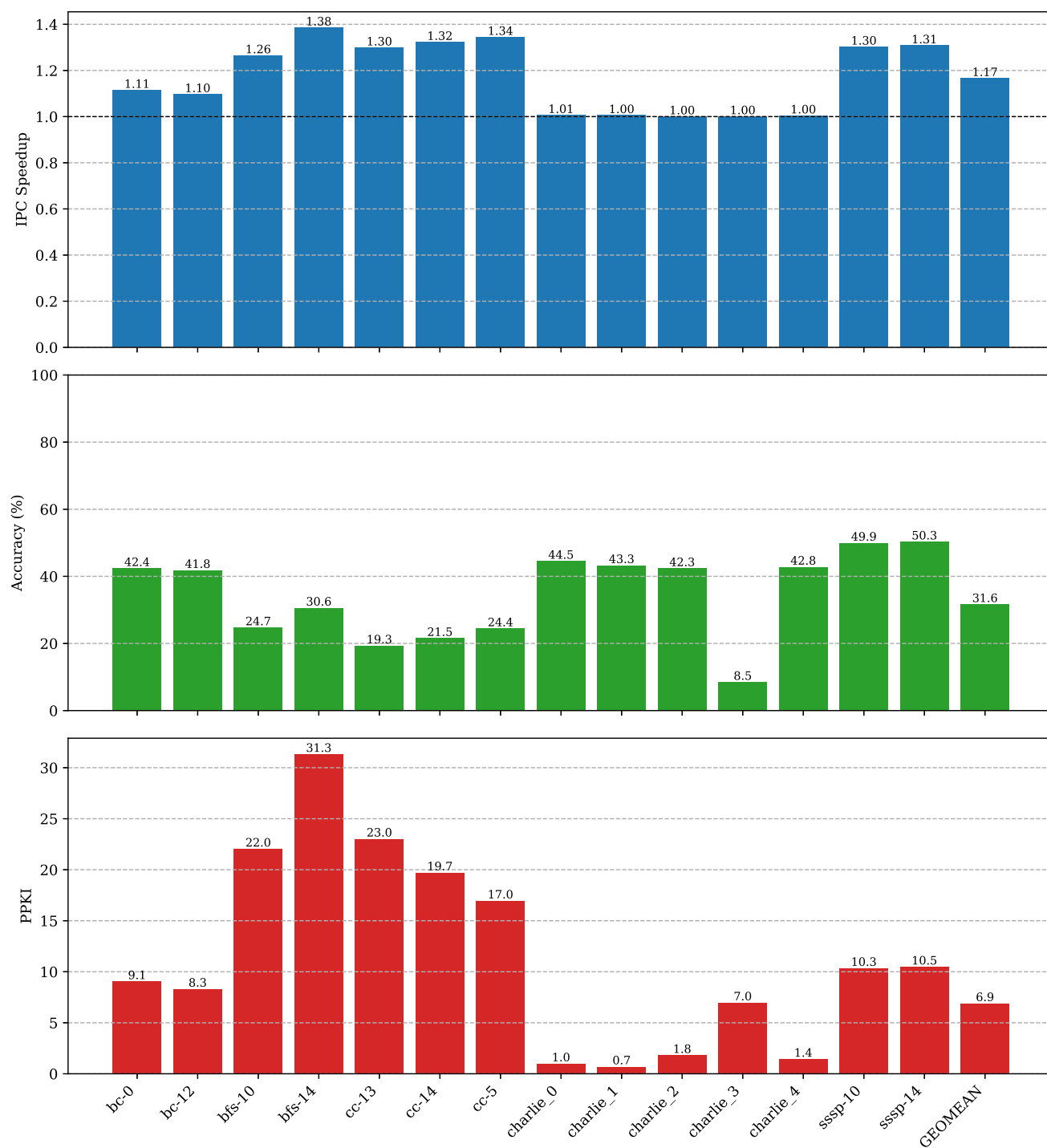
The performance of the system-aware strided GHB prefetcher in both the full-BW and limited-BW systems is shown in fig. 4 and fig. 5, respectively.

**Full Bandwidth System** In the full-BW system, the system-aware prefetcher achieves higher accuracy on most workloads while preserving the IPC speedups observed with the system-unaware prefetcher. In other words, it issues fewer pointless prefetches without sacrificing performance.

**Limited Bandwidth System** In the limited-BW system, the system-aware prefetcher reduces IPC speedups for `bfs-10` and `bfs-14`, but otherwise maintains or improves them relative to the system-unaware baseline. Overall, the geometric mean IPC speedup declines less than with the system-unaware prefetcher under bandwidth constraints.

Prefetch accuracy increases by more than 18% (geometric mean), reflecting stricter issuance and better selectivity. This is consistent with the lower PPKI observed: the system-aware prefetcher issues substantially fewer prefetches, conserving bandwidth for useful demand requests.

Figure 4: Task 3: System-Aware Strided GHB Prefetcher (1 CPU, *Full* BW)

Figure 5: Task 3: System-Aware Strided GHB Prefetcher (1 CPU, *Limited* BW)

## Task 4/4: Design Your Own Prefetcher

In Task 4 I implemented my own prefetcher design.

### My Prefetcher

I based my preliminary design on the Best-Offset-Prefetcher (BOP) that won DPCA2 in 2015 . Considering its success in that competition, I was expecting good performance. The strided GHB prefetcher especially struggled on the Charlie workloads, which lack accesses of constant stride. My BOP is a direct replication of the system described in (Michaud 2016).

As can be seen in fig. 6, the BOP outperforms the system-aware strided GHB on the Charlie workloads and the **bfs-10** and **bfs-14** workloads. On the remaining graph analytics workloads however, the strided GHB prefetcher outperformed the BOP algorithm by a large margin.

Given the complimentary nature of both prefetcher designs, I decided to combine the two prefetcher in a hybrid design. The hybrid design runs a constant tournament between the ghb and bop prefetchers, selecting the better performing one on the fly. The hybrid prefetcher runs instances of either subprefetcher and tracks which cache lines would have been prefetched in a shadow table. On each cache access, the hybrid prefetcher “virtually” runs each prefetcher, updating its state and virtually issuing prefetches. If a virtually issued cache line ends up being useful later on, the hybrid prefetcher increases its preference for the prefetcher that would have issued that useful prefetch. All the while, the hybrid prefetcher actually issues the prefetches suggested by the winning prefetcher.

The hybrid prefetcher works pretty well. The graphs show that the IPC Speedup on most workloads is close to the max speedup of either subprefetcher. The hybrid’s geometric mean IPC speedup is 0.01 higher than system-aware GHB’s. The exception is on the **bfs-10** the hybrid is considerably worse then the constituent subprefetcher’s results.

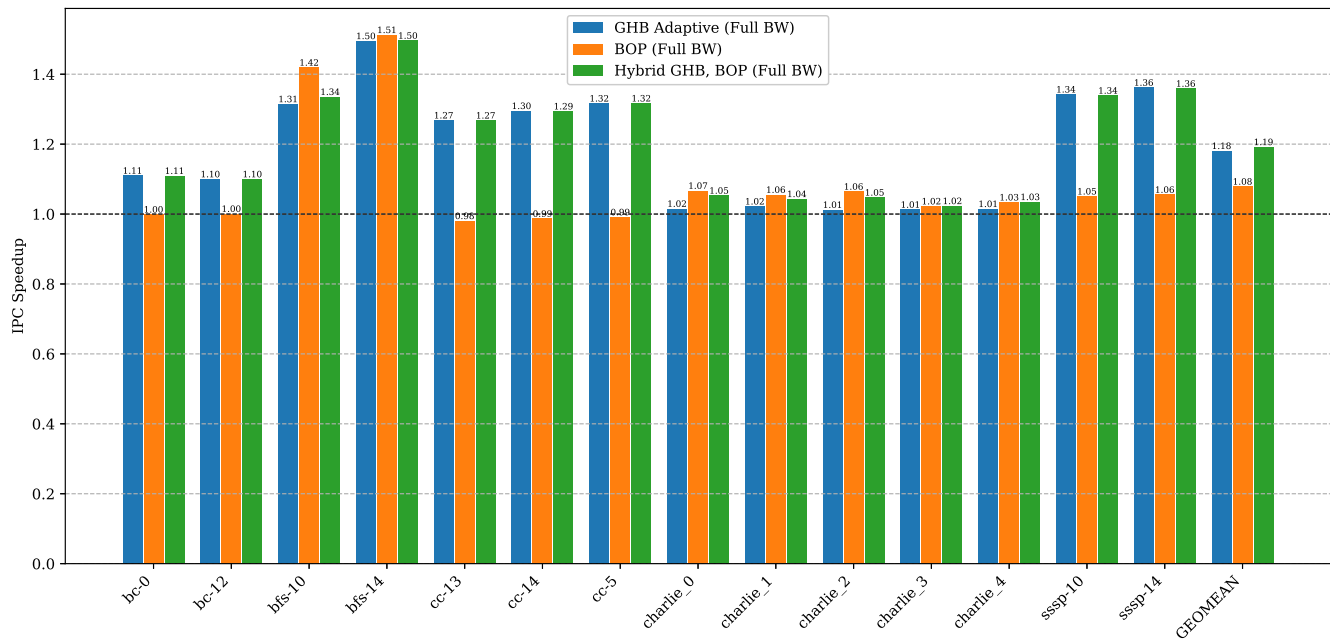


Figure 6: Task 4: Hybrid prefetcher chooses the better performing of either BOP or GHB (1 CPU, Full BW)

### Future Work

I realize that the BOP doesn’t perform particularly

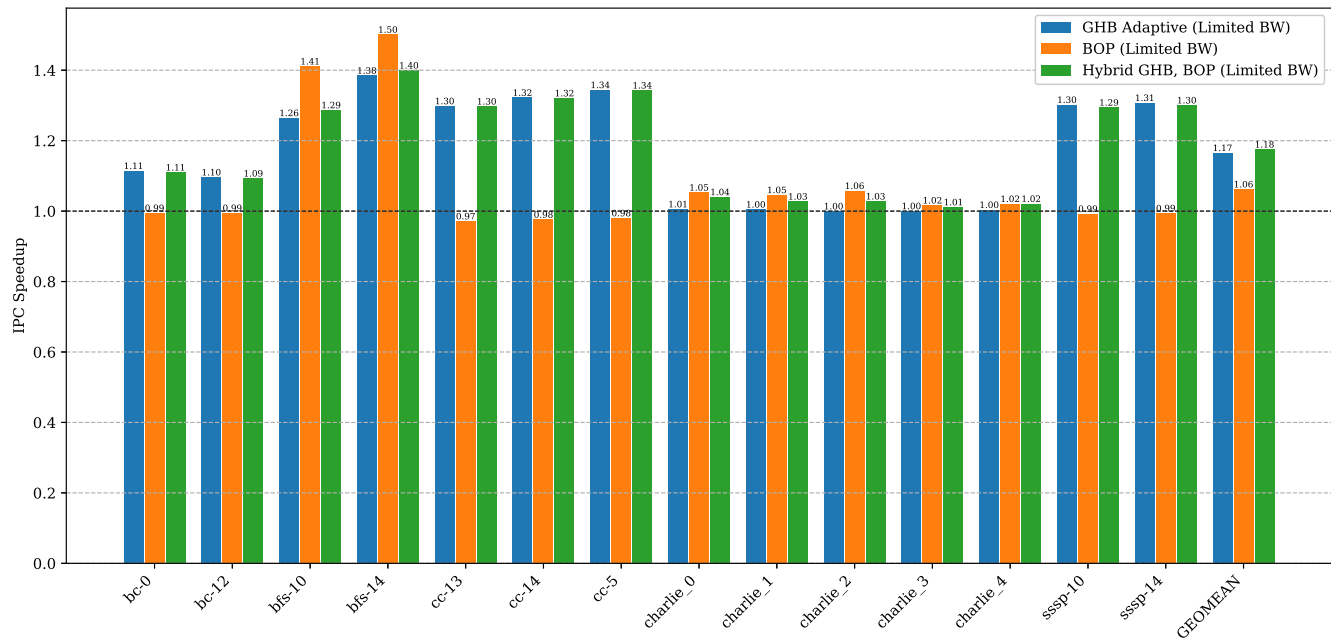
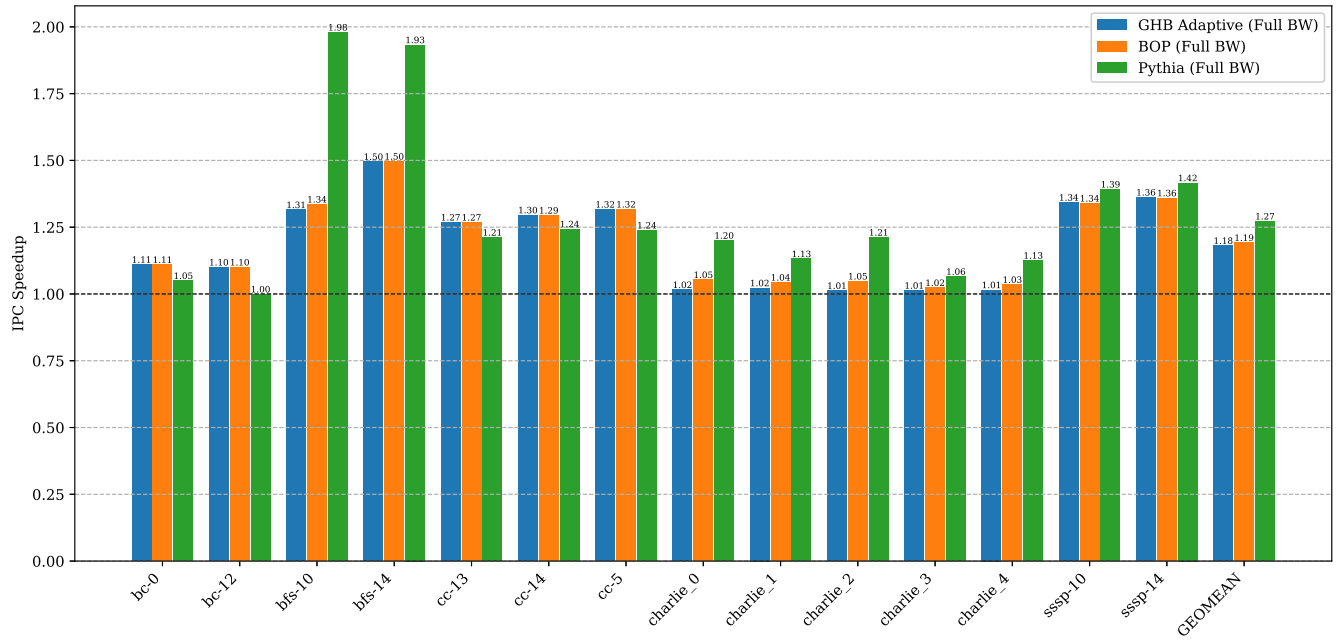
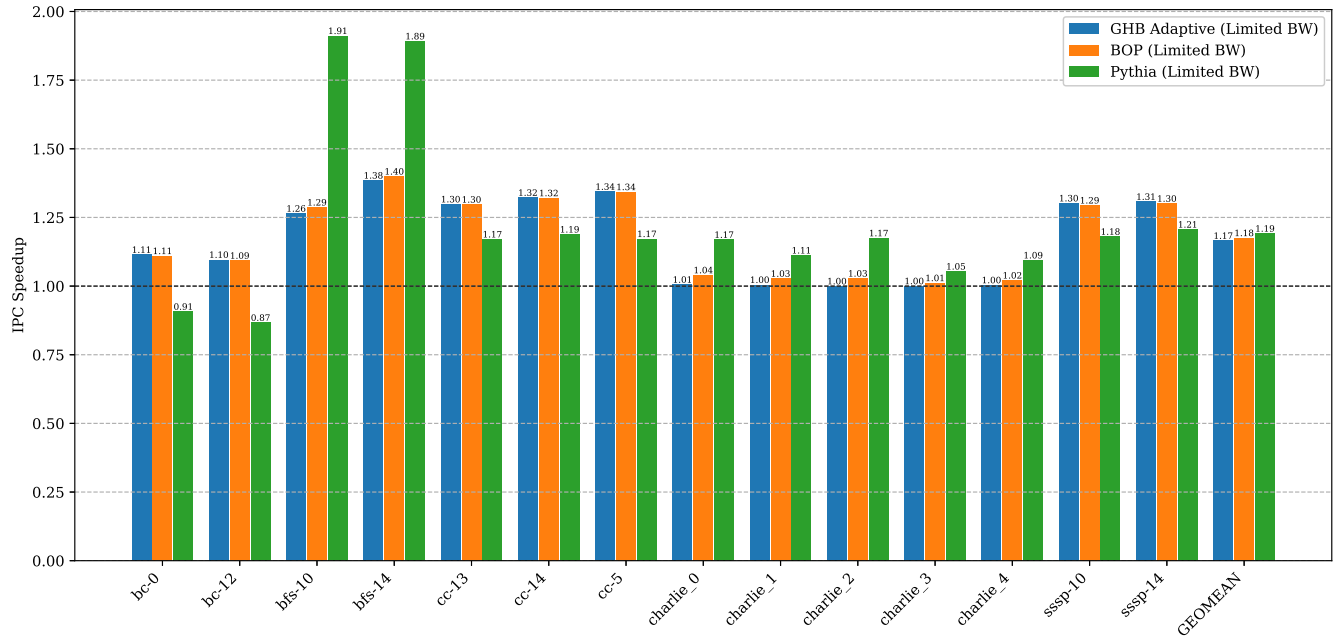


Figure 7: Task 4: Hybrid prefetcher chooses the better performing of either BOP or GHB (1 CPU, *Limited BW*)

### Bonus Task: Comparison Against a State-Of-The-Art Prefetcher

In the bonus task, I compare Pythia (Bera et al. 2021), a state-of-the-art prefetcher against the other prefetchers I designed in this lab.

Figure 8: Bonus Task: IPC Speedup Comparison (1 CPU, *full* BW)Figure 9: Bonus Task: IPC Speedup Comparison (1 CPU, *Limited* BW)

## Citations

- Bera, Rahul, Konstantinos Kanellopoulos, Anant V. Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. 2021. “Pythia: A Customizable Hardware Prefetching Framework Using Online Reinforcement Learning.” *CoRR* abs/2109.12021. <https://arxiv.org/abs/2109.12021>.
- Gober, Nathan, Gino Chacon, Lei Wang, et al. 2022. *The Championship Simulator: Architectural Simulation for Education and Competition*. <https://arxiv.org/abs/2210.14324>.
- Michaud, Pierre. 2016. “Best-Offset Hardware Prefetching.” *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 469–80. <https://doi.org/10.1109/HPCA.2016.7446087>.
- Nesbit, K. J., and J. E. Smith. 2004. “Data Cache Prefetching Using a Global History Buffer.” *10th International Symposium on High Performance Computer Architecture (HPCA’04)*, 96–96. <https://doi.org/10.1109/HPCA.2004.10030>.